MSC2050 Discrete Mathematics, Presentation 5

Dr. Anna Tomskova

Inha University in Tashkent

Spring 2024
Version 1.0 typed under LaTeX

Algorithms

(paragraph 3.1)

- given a sequence of integers, find the largest one;
- given a set, list all its subsets;
- given a set of integers, put them in increasing order;
- given a network, find the shortest path between two vertices

An algorithm is a finite sequence of precise instructions for performing a computation or for solving a problem.

The term algorithm is a corruption of the name al-Khowarizmi, a mathematician of the ninth century, whose book on Hindu numerals is the basis of modern decimal notation.
Originally, the word algorism was used for the rules for performing arithmetic using decimal notation.
Algorism evolved into the word algorithm by the eighteenth century.
With the growing interest in computing machines, the concept of an algorithm was given a more general meaning, to include all definite procedures for solving problems.

Describe an algorithm for finding the maximum (largest) value in a finite sequence of integers.

We perform the following steps.
1. Set the temporary maximum equal to the first integer in the sequence. (The temporary maximum will be the largest integer examined at any stage of the procedure.)
2. Compare the next integer in the sequence to the temporary maximum, and if it is larger than the temporary maximum, set the temporary maximum equal to this integer.
3. Repeat the previous step if there are more integers in the sequence.
4. Stop when there are no integers left in the sequence. The temporary maximum at this point is the largest integer in the sequence.

We will use pseudocode to write algorithms.
Pseudocode - intermediate step between English language and programming language.

Problem from the past midterm paper

Let P and Q be bit strings of length n. Devise an **algorithm** which calculates

$$d = \text{the amount of positions where } P \text{ is different from } Q.$$

Solution: Assume that $P = p_1 p_2 \dots p_n$ and $Q = q_1 q_2 \dots q_n$ are the given bit strings.
The question asks to count how many times p_i is different from q_i. Hence, one loop is necessary to go through all possible $1 \leq i \leq n$ and compare p_i with q_i.
We will need to introduce a counting variable - d.

procedure *compare bit strings* (P, Q: bit strings, n: positive integer)
d:=0
for $i := 1$ **to** n
    if $p_i \neq q_i$ **then** d:=d+1
return d

Answer Q1

Properties of algorithm

- **Input.** An algorithm has input values from a specified set.
- **Output.** From each set of input values an algorithm produces output values from a specified set. The output values are the solution to the problem.
- **Definiteness.** The steps of an algorithm must be defined precisely.
- **Correctness.** An algorithm should produce the correct output values for each set of input values.
- **Finiteness.** An algorithm should produce the desired output after a finite (but perhaps large) number of steps for any input in the set.
- **Effectiveness.** It must be possible to perform each step of an algorithm exactly and in a finite amount of time.
- **Generality.** The procedure should be applicable for all problems of the desired form, not just for a particular set of input values.

Searching Algorithms

The problem

Locate an element x in a list of distinct elements $a_1, a_2, \ldots, a_n$, or determine that it is not in the list.
The solution to this search problem is i if $x = a_i$ and is 0 if x is not in the list.

Linear search

procedure linear search(x:integer, $a_1, a_2, \ldots, a_n$: distinct integers)
$i := 1$
while($i \leq n$ and $x \neq a_i$)
    $i := i + 1$
if $i \leq n$ **then** $location := i$
else $location := 0$
return $location$

Searching Algorithms

The Binary Search Algorithm

procedure binary search(x:integer, $a_1, a_2, \ldots, a_n$: increasing integers)
$i := 1$ i is left endpoint of search interval
$j := n$ i is right endpoint of search interval
while$(i < j)$
    $m := \lfloor (i + j)/2 \rfloor$
    if $x > a_m$ **then** $i := m + 1$
    else $j := m$
if $x = a_i$ **then** $location := i$
else $location := 0$
return $location$

The Halting Problem

The halting problem asks whether there is a procedure that does this:
It takes as input a computer program and input to the program and determines whether the program will eventually stop when run with this input.

In 1936 Alan Turing showed that no such procedure exists!!!
Proof: Assume that there exists a program $Halt(P, I)$ that solves the halting problem, $Halt(P, I)$ returns $TRUE$ if P halts on I and $FALSE$ if P goes into an infinite loop on I.
The given this program for the Halting Problem, we could construct the following string/code Z:

Procedure $Z(x$:string)
If $Halt(x, x)$ **then** loop forever
else halt.

Case 1: Program Z halts on input Z. Hence, $Halt(Z, Z) = TRUE$. Hence, program Z loops forever on input Z. Contradiction!
Case 2: Program Z loops forever on input Z. Hence, $Halt(Z, Z) = FALSE$. Hence, program Z halts on input Z. Contradiction!

Homework:

- Solve *odd* exercises starting at p. 202, #1-#33.

The Growth of Functions

(Paragraph 3.2)

big-O notation- to compare algorithms

- The time required to solve a problem depends on the number of operations it uses.
- The time also depends on the hardware and software used to run the program that implements the algorithm.
- On a supercomputer we might be able to solve a problem of size n a million times faster than we can on a PC.
- This factor of one million will not depend on n. This is a constant.

Big-O notation allows:

- to estimate the growth of a function without worrying about constant multipliers or smaller order terms. So we do not have to worry about the hardware and software used to implement an algorithm.
  So we can estimate the number of operations an algorithm uses as its input grows.
- to compare two algorithms to determine which is more efficient as the size of the input grows.

Definition

Let f and g be functions from $\mathbb{R}$ to $\mathbb{R}$.

We say that $f(x)$ is $O(g(x))$ if there are constants C and k such that

$$|f(x)| \leq C|g(x)|$$

whenever $x > k$. We read "$f(x)$ is big-o of $g(x)$." We write $f(x) = O(g(x))$.

Intuitively, the definition that $f(x)$ is $O(g(x))$ says that $f(x)$ grows slower than some fixed multiple of $g(x)$ as x grows without bound.

The constants C and k in the definition of big-O notation are called **witnesses** to the relationship $f(x)$ is $O(g(x))$.
To establish that $f(x)$ is $O(g(x))$ we need only one pair of witnesses to this relationship.
When there is one pair of witnesses to the relationship $f(x)$ is $O(g(x))$, there are infinitely many pairs of witnesses.

Answer Q2

The history of big-O notation

Big-O notation has been used in mathematics for more than a century.

In computer science it is widely used in the analysis of algorithms.

The German mathematician Paul Bachmann first introduced big-O notation in 1892 in an important book on number theory.

The big-O symbol is sometimes called a Landau symbol after the German mathematician Edmund Landau, who used this notation throughout his work.

The use of big-O notation in computer science was popularized by Donald Knuth, who also introduced the big-Ω and big-Θ notations defined later.

Examples

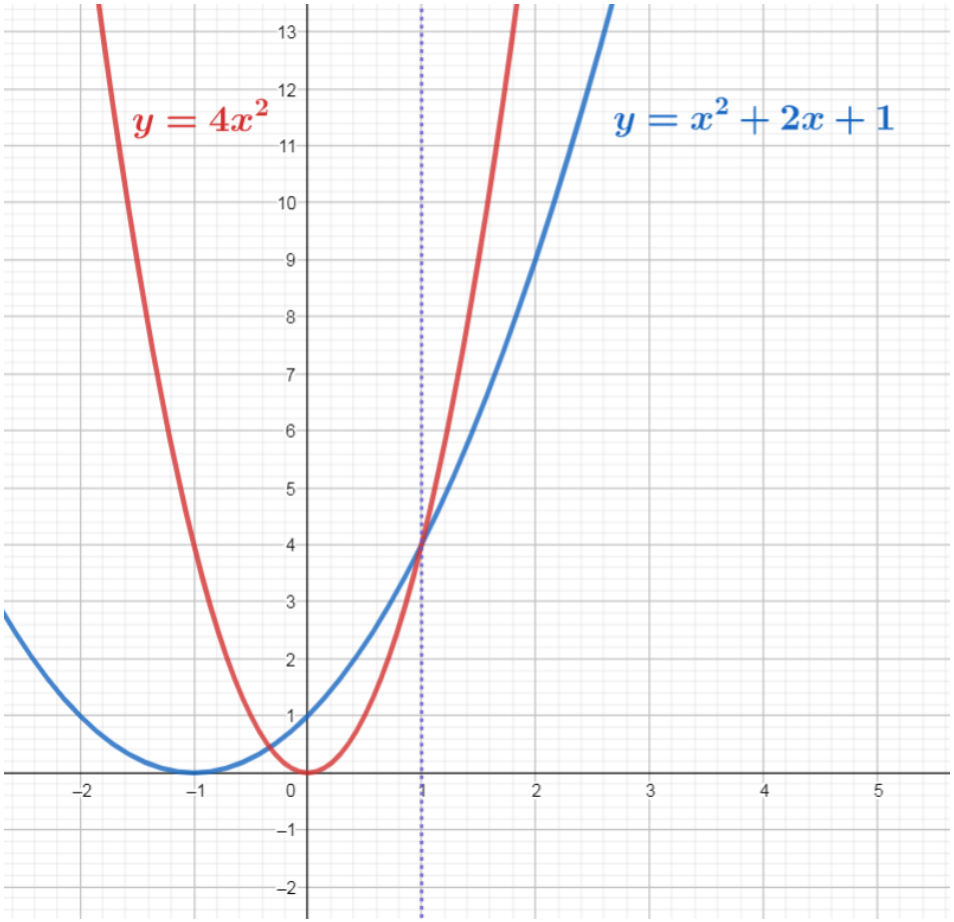
Show that $f(x) = x^2 + 2x + 1$ is $O(x^2)$.

We observe that we can readily estimate the size of $f(x)$ when $x > 1$
because $x < x^2$ and $1 < x^2$ when $x > 1$.

It follows that whenever $x > 1$, we have

$$x^2+2x+1 \leq x^2+2x^2+x^2 = 4x^2.$$

Consequently, we can take $C = 4$ and $k = 1$ as witnesses to show that $f(x)$ is $O(x^2)$.



Examples

Show that $f(x) = x^2 + 2x + 1$ is $O(x^2)$.

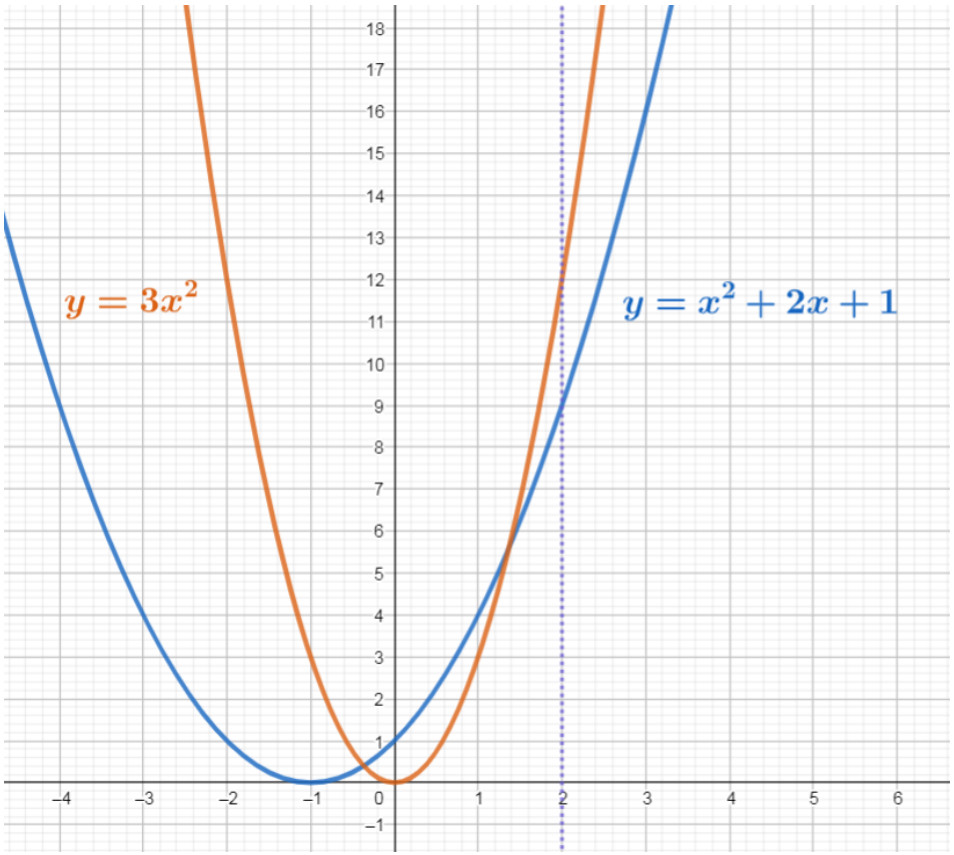
Another way is to obtain the estimate when $x > 2$,
we have $2x < x^2$ and $1 < x^2$.

Consequently, if $x > 2$, we have

$$x^2 + 2x + 1 < x^2 + x^2 + x^2 = 3x^2.$$

It follows that $C = 3$ and $k = 2$ are also witnesses to the relation $f(x)$ is $O(x^2)$.

Answer Q3

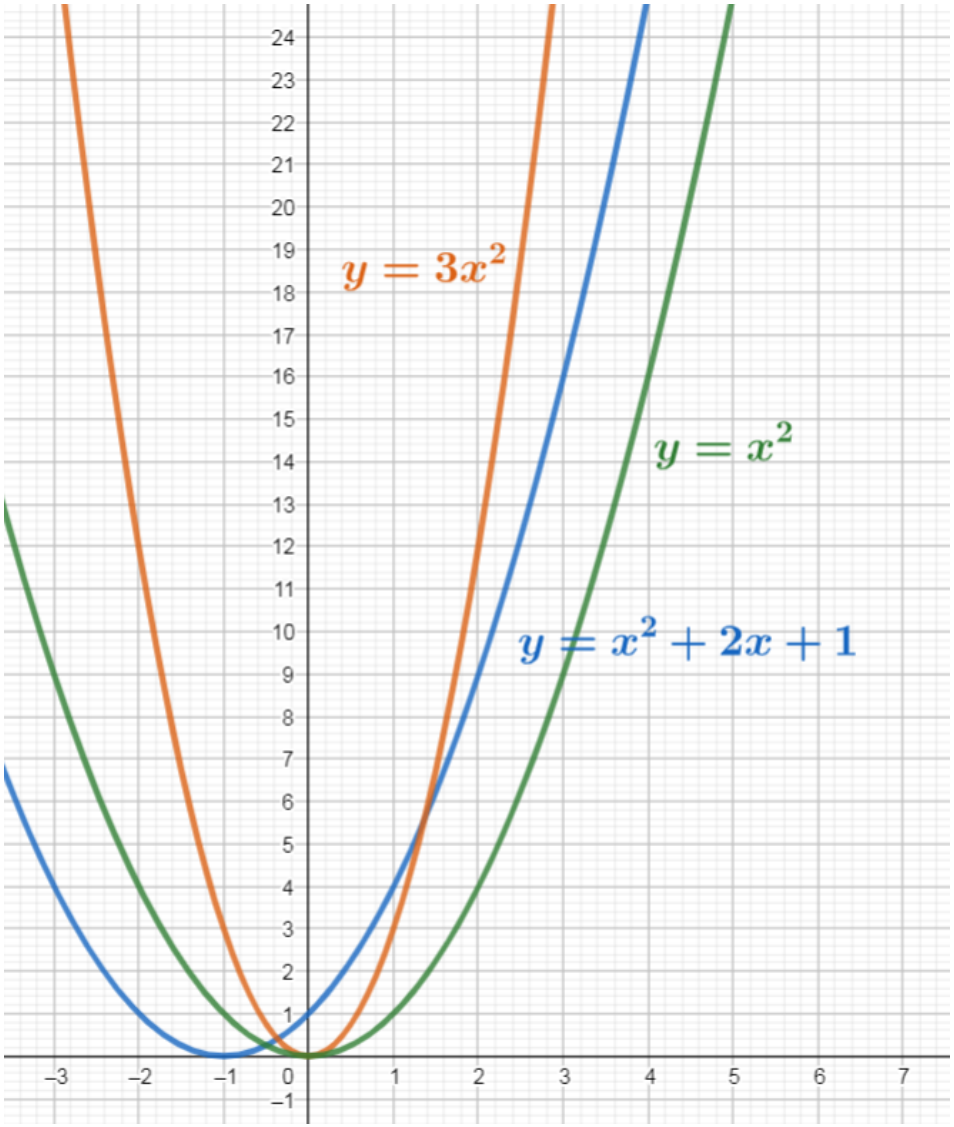


Functions of the same order

It is also true that x^2 is $O(x^2 + 2x + 1)$,
because whenever $x > 1$, we have $x^2 < x^2 + 2x + 1$.

This means that $C = 1$ and $k = 1$ are witnesses to the relationship x^2 is $O(x^2 + 2x + 1)$.

Hence, $x^2 + 2x + 1 = O(x^2)$ and $x^2 = O(x^2 + 2x + 1)$.



If $f(x) = O(g(x))$ and $g(x) = O(f(x))$, then f and g are of the **same order**.

The best estimate

If $|f(x)| \leq C_1|g(x)|$ for $x > k$, and if $|g(x)| \leq C_2|h(x)|$ for $x > k$, then

$$|f(x)| \leq C|h(x)|, \quad x > k.$$

Hence, $f(x) = O(h(x))$.

When big-O notation is used, the function g in the relationship $f(x)$ is $O(g(x))$ is chosen to be as small as possible.

For example, in the estimate

$$x^2 + 2x + 1 < x^2 + x^2 + x^2 = 3x^2.$$

we could write

$$x^2 + 2x + 1 < x^3 + x^3 + x^3 = 3x^3.$$

but we estimate as good as we can.

Big-O relationship does NOT hold

Show that n^2 is not $O(n)$.

To show that n^2 is not $O(n)$, we must show that NO pair of witnesses C and k exist such that

$$n^2 \leq Cn$$

whenever $n > k$.
We will use a proof by contradiction to show this.

Suppose that there are constants C and k for which $n^2 \leq Cn$ whenever $n > k$. Observe that when $n > 0$ we can divide both sides of the inequality by n to obtain the equivalent inequality $n \leq C$.

For fixed values of k and C, take n to be larger than maximum of k and C, it is not true that $n \leq C$ even though $n > k$.

This contradiction shows that n^2 is not $O(n)$.

Big-O Estimates for Some Important Functions

Theorem

Let

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x + a_0,$$

where $a_0, a_1, \ldots, a_{n-1}, a_n$ are real numbers. Then $f(x)$ is $O(x^n)$.

Proof: Using the triangle inequality, if $x > 1$ we have

$$\begin{aligned} |f(x)| &= |a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x + a_0| \\ &\leq |a_n| x^n + |a_{n-1}| x^{n-1} + \ldots + |a_1| x + |a_0| \\ &= x^n (|a_n| + |a_{n-1}|/x + \ldots + |a_1|/x^{n-1} + |a_0|/x^n) \\ &\leq x^n (|a_n| + |a_{n-1}| + \ldots + |a_1| + |a_0|). \end{aligned}$$

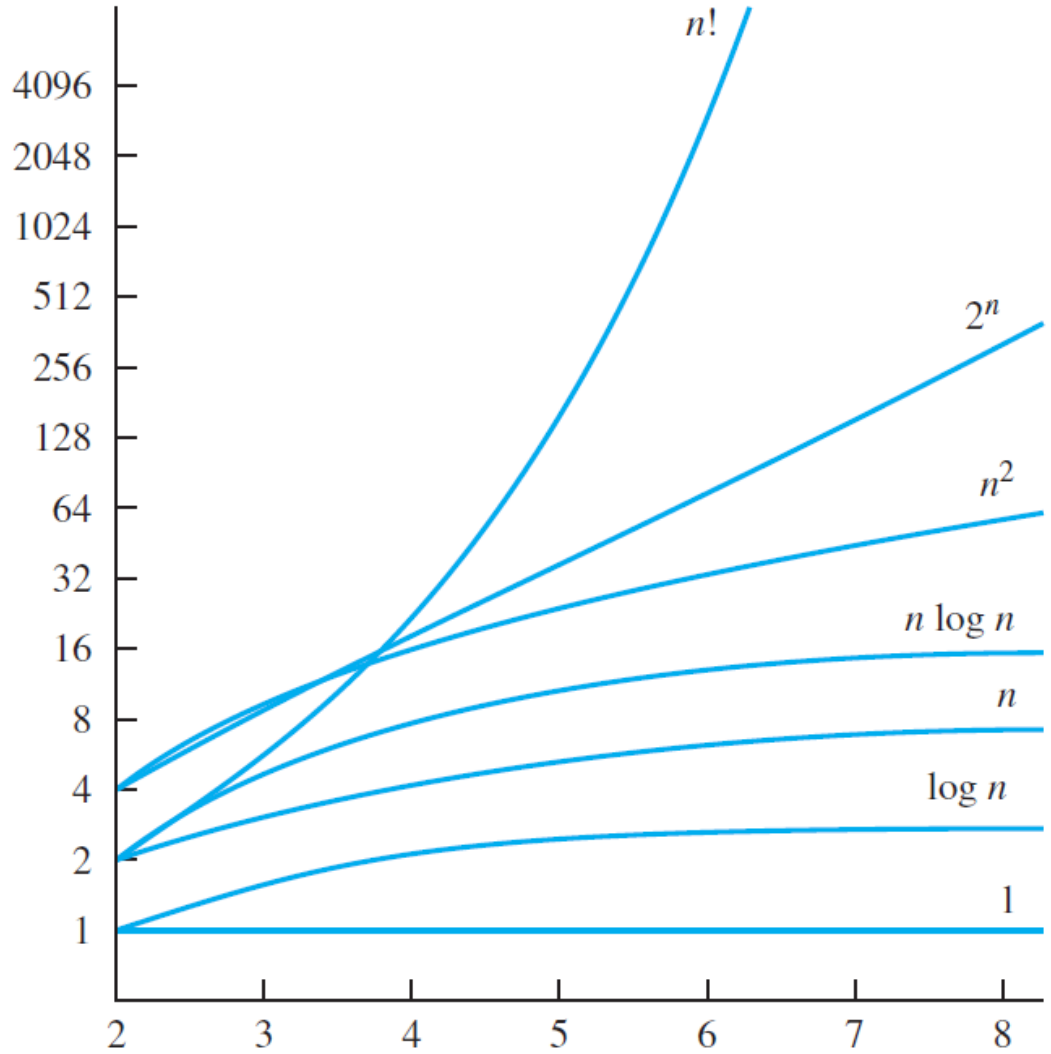
This shows that $|f(x)| \leq C x^n$, where

$$C = |a_n| + |a_{n-1}| + \ldots + |a_0|$$

whenever $x > 1$.

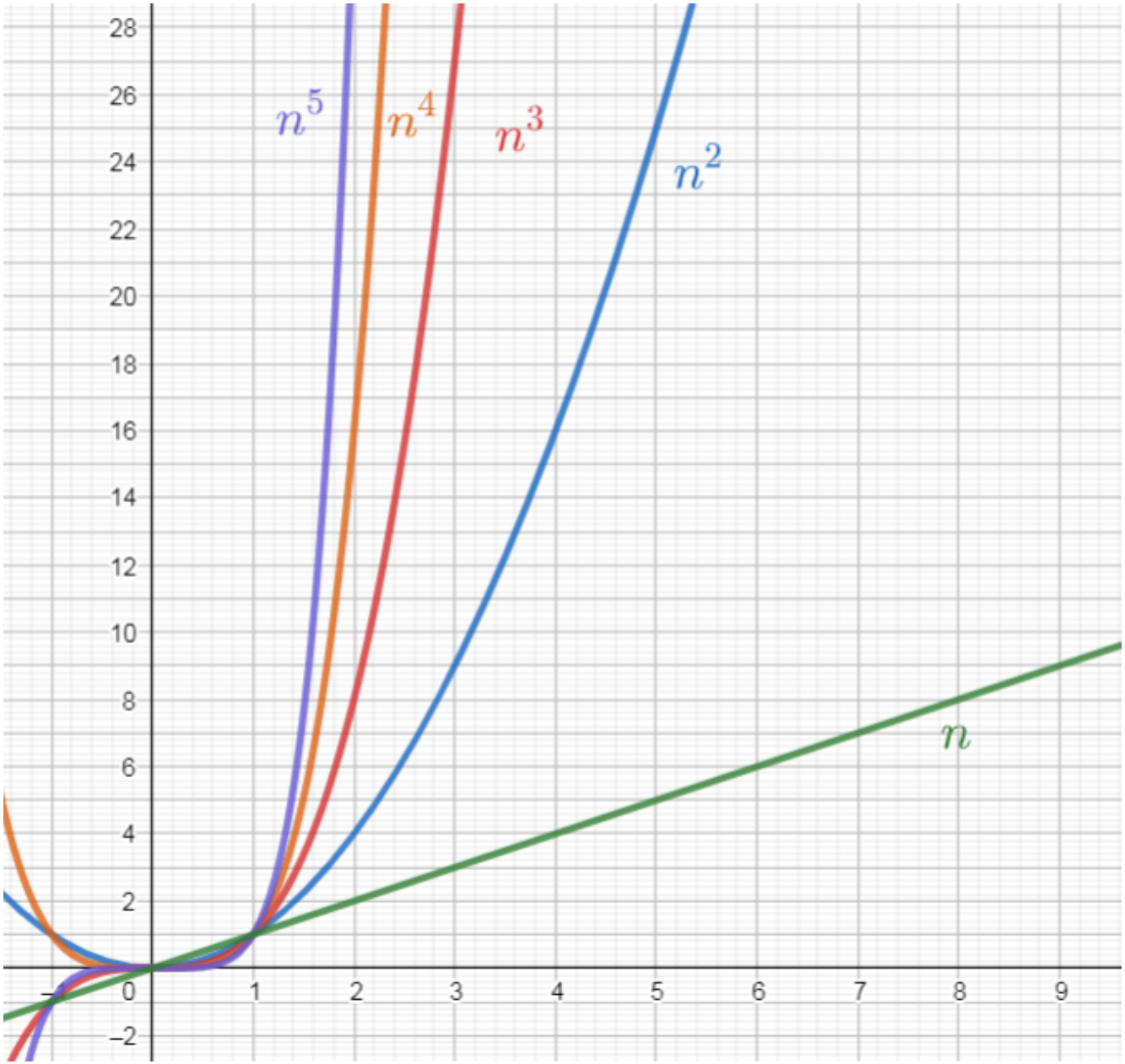
For large enough n we have

$$1 < \log n < n < n \log n < n^2 < 2^n < n!$$



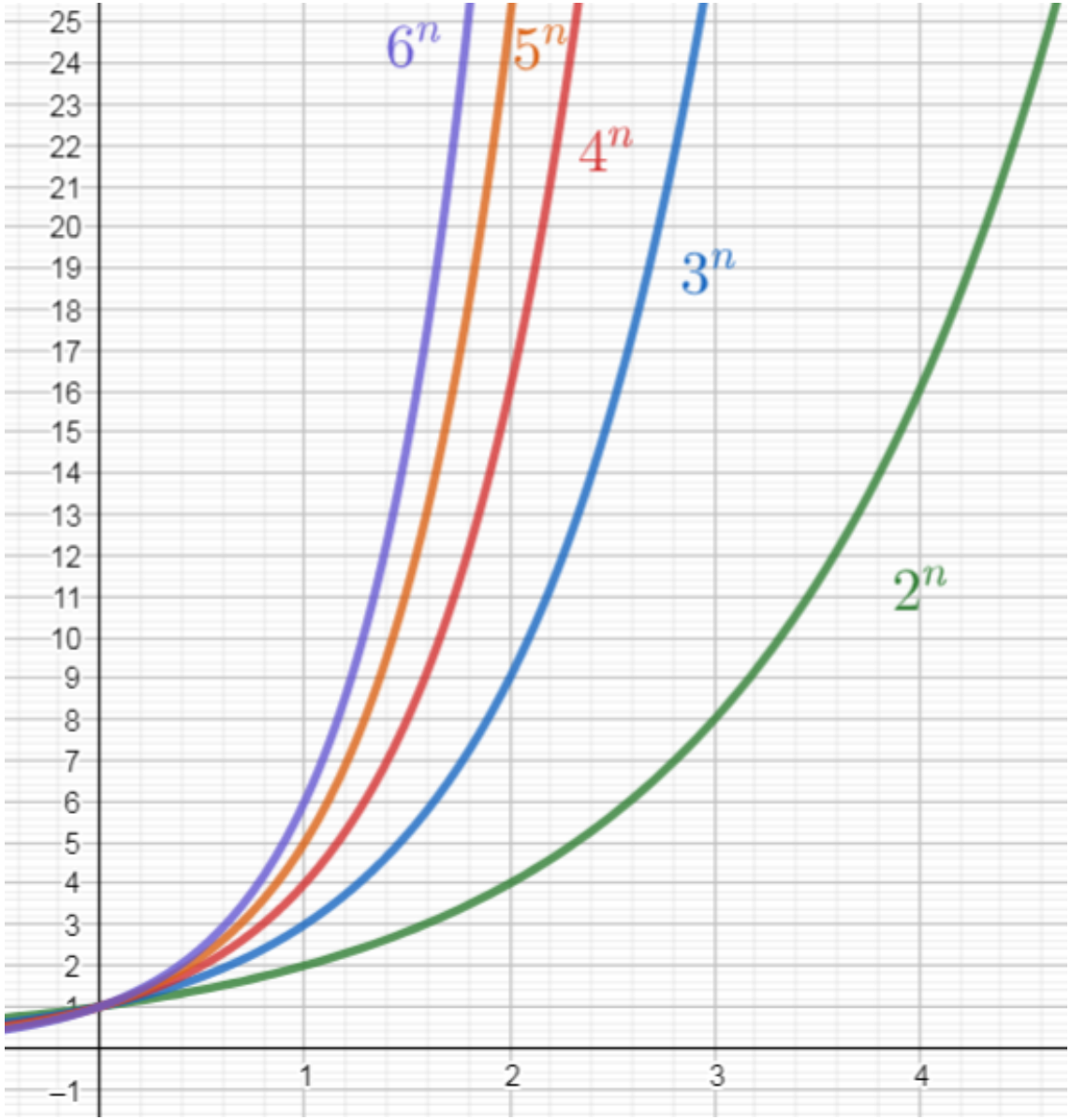
If $d > c > 1$, then

$$n^c = O(n^d) \text{ but } n^d \neq O(n^c)$$



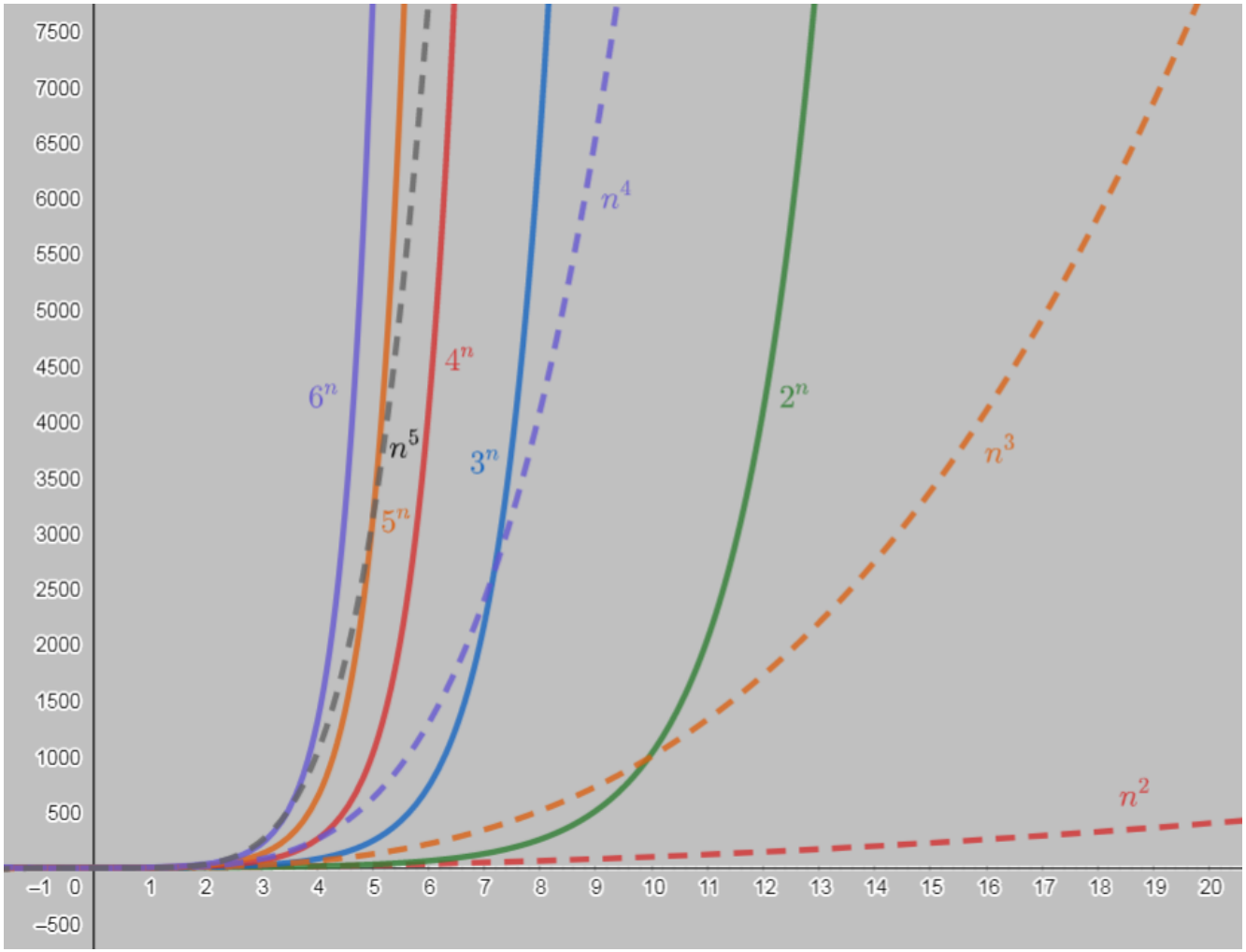
If $c > b > 1$, then

$$b^n = O(c^n) \text{ but } c^n \neq O(b^n)$$



If $d > 0$ and $b > 1$

$$n^d = O(b^n) \text{ but } b^n \neq O(n^d)$$



Answer Q4

The growth of combinations of functions

Theorem

$K = O(1)$

Theorem

If $f(x) = O(g(x))$, then $Kf(x) = O(g(x))$.

Theorem

Suppose that $f_1(x)$ is $O(g_1(x))$ and that $f_2(x)$ is $O(g_2(x))$. Then $(f_1 + f_2)(x)$ is $O(max(|g_1(x)|, |g_2(x)|))$.

Corollary

Suppose that $f_1(x)$ and $f_2(x)$ are both $O(g(x))$. Then $(f_1 + f_2)(x)$ is $O(g(x))$.

Theorem

Suppose that $f_1(x)$ is $O(g_1(x))$ and $f_2(x)$ is $O(g_2(x))$. Then $(f_1 f_2)(x)$ is $O(g_1(x)g_2(x))$.

Example

Give a big-O estimate for $f(x) = (x+1)\log(x^2+1) + 3x^2$.

Solution: First, a big-O estimate for $(x+1)\log(x^2+1)$ will be found.
Note that $(x+1)$ is $O(x)$.
Furthermore, $x^2+1 \leq 2x^2$ when $x > 1$.
Hence, if $x > 2$, then

$$\log(x^2+1) \leq \log(2x^2) = \log 2 + \log x^2 = \log 2 + 2\log x \leq 3\log x.$$

This shows that $\log(x^2+1)$ is $O(\log x)$.
It follows that

$$(x+1)\log(x^2+1) = O(x\log x).$$

Because $3x^2$ is $O(x^2)$, we have that $f(x)$ is $O(\max(x\log x, x^2))$.
Because $x\log x \leq x^2$, for $x > 1$, it follows that $f(x)$ is $O(x^2)$.

More examples

$f(n) = 7n^2 + 3n \log n + 5n + 1000$ is $O(n^2)$

$g(n) = 7n^4 + 3^n + 10000000$ is $O(3^n)$

$h(n) = 7n(n + \log n)$ is $O(n^2)$.

Answer Q5

Application in algorithms

Suppose that you have two different algorithms for solving a problem. To solve a problem of size n, the first algorithm uses exactly $n^2 2^n$ operations and the second algorithm uses exactly $n!$ operations. As n grows, which algorithm uses fewer operations?

Answer: The first algorithm.

Big-Omega and Big-Theta Notation

We say that $f(x) = \Omega(g(x))$ if there are positive constants C and k such that

$$|f(x)| \geq C|g(x)|$$

whenever $x > k$. We read "$f(x)$ is big-Omega of $g(x)$."

$f(x) = \Omega(g(x))$ is the same as $g(x) = O(f(x))$.

We say that $f(x) = \Theta(g(x))$ if $f(x) = O(g(x))$ and $f(x) = \Omega(g(x))$. When $f(x) = \Theta(g(x))$ we say that f is big-Theta of $g(x)$ or $f(x)$ is of order $g(x)$, or that $f(x)$ and $g(x)$ are of the same order.

$f(x) = \Theta(g(x))$ is the same as $f(x) = O(g(x))$ and $g(x) = O(f(x))$.
$f(x) = \Theta(g(x))$ if and only if $g(x) = \Theta(f(x))$.

Answer Q6

Examples

Theorem

Let

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x + a_0,$$

where $a_0, a_1, \ldots, a_{n-1}, a_n$ are real numbers. Then $f(x)$ is of order x^n.

Big-O in Calculus

$f(x) = O(g(x))$ if and only if $\lim_{n\to\infty} \frac{f(x)}{g(x)}$ is a constant or zero.

$f(x) = \Theta(g(x))$ if and only if $\lim_{n\to\infty} \frac{f(x)}{g(x)}$ is a non-zero constant.

Proof of the theorem:

$$\lim_{n\to\infty} \frac{a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x + a_0}{x^n}$$

$$= \lim_{n\to\infty} \left(a_n + \frac{a_{n-1}}{x} + \ldots + \frac{a_1}{x^{n-1}} + \frac{a_0}{x^n} \right) = a_n.$$

Homework:

- Solve *odd* exercises starting at p. 216 till #49.
- For those who know Calculus (at least remember what limit is) solve *odd* exercises starting at p. 216 till the end.

Complexity of Algorithms

(Paragraph 3.3)

Desired Properties of a Good Algorithm

How efficient is an algorithm or piece of code?

Any good algorithm should satisfy 2 obvious conditions:

1. compute correct (desired) output (for the given problem)
2. be effective (fast)

ad. 1) correctness of algorithm
ad. 2) complexity of algorithm

Complexity of algorithm measures how fast is the algorithm (time complexity) and what amount of memory it uses (space complexity) - time and memory - 2 basic resources in computations

The speed of algorithm

How to measure how fast (or slow) an algorithm is?

There are 2 issues to be considered when designing such a measure:

- independence on any programming language and hardware/software platform
- maximum independence on particular input data It should be an internal property of the algorithm itself

The idea: count basic operations of the algorithm

Dominating Operations

Simplification: it is not necessary to count all the operations It is enough to count the representative ones.

Before doing a complexity analysis 2 steps must be done:

1. Determine the dominating operation set. Dominating operations are those which cover the amount of work which is proportional to the whole amount of work of the algorithm
2. Observe what (in input) influences the number of dominating operations (data size)

Example - determining dominating operations

What can be the dominating operation set in the following algorithm?

procedure *maximum*($a_1, a_2, \ldots, a_n$: integers)
$max := a_1$
for $i := 2$ **to** n
    if $max < a_i$ **then** $max := a_i$
return max

assignment $max := a_1$? no
index increment in the loop ? yes
comparison $max < a_i$? yes
assignment $max := a_i$? yes
return max ? no

Example - determining the data size

What is the data size in the following algorithm?

procedure *maximum*($a_1, a_2, \ldots, a_n$: integers)
$max := a_1$
for $i := 2$ **to** n
    if $max < a_i$ **then** $max := a_i$
return max

Data size: length of the sequence $a_1, a_2, \ldots, a_n$ is n

Having determined the dominating operation and data size we can determine time complexity of the algorithm

Time Complexity of Algorithm

Definition

Time Complexity of Algorithm is the number of dominating operations executed by the algorithm as the function of data size.

Time complexity measures the amount of work done by the algorithm during solving the problem in the way which is independent on the implementation and particular input data.

The lower time complexity the faster algorithm!

Attempt to find time complexity

procedure *maximum*($a_1, a_2, \ldots, a_n$: integers)
$max := a_1$
for $i := 2$ **to** n
    if $max < a_i$ **then** $max := a_i$
return max

index increment in the loop $n - 1$ times

comparison $max < a_i$ $n - 1$ times

assignment $max := a_i$??? depends on the data:
If the input is 5,4,3,2,1, then 0
If the input is 1,2,3,4,5, then $n - 1$

Worst-case Complexity

By the worst-case performance of an algorithm, we mean the largest number of operations needed to solve the given problem using this algorithm on input of specified size.
Worst-case analysis tells us how many operations an algorithm requires to guarantee that it will produce a solution.

procedure *maximum*($a_1, a_2, \ldots, a_n$: integers)
$max := a_1$
for $i := 2$ **to** n
    if $max < a_i$ **then** $max := a_i$
return max

The worst-case performance is
index increment in the loop $n - 1$ times
comparison $max < a_i$ $n - 1$ times
assignment $max := a_i$ $n - 1$ times

Overall we have $3n - 3$ operations, which is $\Theta(n)$ worst-case complexity.

Average-case Complexity

Average-case complexity is the average number of operations used to solve the problem over all possible inputs of a given size.

procedure *maximum*($a_1, a_2, \ldots, a_n$: integers)
$max := a_1$
for $i := 2$ **to** n
    if $max < a_i$ **then** $max := a_i$
return max

The average-case performance is
index increment in the loop $n - 1$ times
comparison $max < a_i$ $n - 1$ times
assignment $max := a_i$

$$\frac{1 + 2 + \ldots + n - 1}{n} = \frac{(1 + n - 1)(n - 1)}{n} = n - 1$$

Overall we have $3n - 3$ operations, which is $\Theta(n)$ average-case complexity.

Example-linear search

The problem

Locate an element x in a list of distinct elements $a_1, a_2, \ldots, a_n$, or determine that it is not in the list.
The solution to this search problem is i if $x = a_i$ and is 0 if x is not in the list.

Algorithm

procedure linear search(x:integer, $a_1, a_2, \ldots, a_n$: distinct integers)
$i := 1$
while($i \leq n$ and $x \neq a_i$)
    $i := i + 1$
if $i \leq n$ **then** $location := i$
else $location := 0$
return $location$

Question: Describe the worst-case and average-case performance of the linear search algorithm.
Answer: Average-case complexity: $\Theta(n)$ Worst-case complexity: $\Theta(n)$

More examples

Worst-case complexity of

Searching algorithms:
Linear search $\Theta(n)$
Binary search is $\Theta(\log n)$

Sorting algorithms:
Bubble sort is $\Theta(n^2)$
Insertion sort is $\Theta(n^2)$
Later we will see that the most efficient sorting algorithm is $\Theta(n \log n)$

Matrix multiplication:
Directly from definition $\Theta(n^3)$
The most efficient known one is $\Theta(n^{\sqrt{7}})$

Terminology and tractability

Commonly Used Terminology for the Complexity of Algorithms

Complexity	*Terminology*
$\Theta(1)$	Constant complexity
$\Theta(\log n)$	Logarithmic complexity
$\Theta(n)$	Linear complexity
$\Theta(n \log n)$	Linearithmic complexity
$\Theta(n^b)$	Polynomial complexity
$\Theta(b^n)$, where $b > 1$	Exponential complexity
$\Theta(n!)$	Factorial complexity

A problem that is solvable using an algorithm with polynomial worst-case complexity is called **tractable** (of class P).
Intractable problems cannot be solved using an algorithm with worst-case polynomial time complexity (average-case complexity can be used and for daily life inputs can be fine).

P versus NP

Problems with no algorithm with polynomial worst-case time complexity, for which a solution can be checked in polynomial time are said to belong to the class NP

P versus NP problem

NP=P?

It is one of the seven famous Millennium Prize Problems, of which six remain unsolved. A prize of $1,000,000 is offered by the Clay Mathematics Institute for its solution.

NP-complete problem is the problem in NP such that every problem in NP can be reduced to this NP-complete problem in polynomial time.

SAT (satisfiability) is the problem of determining if there exists an interpretation that satisfies a given Boolean formula.
Only algorithms with exponential worst-case complexity are known for it.
SAT was the first known NP-complete problem, as proved by Stephen Cook in 1971 and independently by Leonid Levin in 1973.

Homework:

- Solve *odd* exercises starting at p. 229, #1-#33.

Answer Q7 and Q8